

Dagster Web Server

Description

- A CLI and web UI to work with Dagster.

Installation and Setup

1. Install the package via `pip`.

```
pip install dagster-webserver
```

2. Generate a Dagster project through the CLI.

- Replace `project_name` with the name of your project.

```
dagster project scaffold --name project_name
```

3. Navigate to the project.

```
cd project_name
```

4. Start the web UI.

```
dagster dev
```

Pipeline Setup

1. Within your `project_name`, find the inner directory with the same name.
2. Within the directory, find the file called `definitions.py`.
3. Within `definitions.py`, setup the following:

- operations: `@op`
- resources: `@resource`
- job: `@job`

4. At the bottom of the file, set a variable called `defs` which instantiates the `Definitions` class with the function input to the `@job` decorator.

Example Script

- `definitions.py`

```
import pandas as pd
from dagster import Definitions, job, op, resource, StringSource
from sqlalchemy import create_engine

@resource(config_schema={"postgres_url": StringSource})
def pg_db(context):
    return create_engine(context.resource_config["postgres_url"])

@op(required_resource_keys={"pg_db"})
def extract_table(context) -> pd.DataFrame:
    return pd.read_sql_table(table_name='sales-dataset', schema='dea_imtiaz_a', con=context.get_resource('pg_db'))

@op
def transform_column(df):
    df["Order ID"] = df["Order ID"].str.lower()
    return df

@op
def write_csv(df):
    df.to_csv("output.csv", index=False)

@job(resource_defs={"pg_db": pg_db})
def postgres_to_csv_job():
    write_csv(transform_column(extract_table()))

defs = Definitions(jobs=[postgres_to_csv_job])
```

5. Setup a `config.yaml` file to store your resource context.

- Place the file in the root of the project scaffold.

```
resources:
  pg_db:
    config:
      postgres_url:
        env: DAGSTER_POSTGRES_URL
```

6. Run the web UI.

```
dagster dev
```

7. Within the `Runs` tab, initialize a new Run.

8. Select `Launchpad`.

9. If you run into a context error, copy and paste the contents of the `config.yaml` file into the editor.

10. Click on `Launch Run` on the bottom right of the UI.

11. Inspect the logs to ensure the pipeline materializes all the steps properly.